Skill - 23

```
himasree@LAPTOP-41COT5AA:~/OSLab$ nano combined_pipeline.c
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc combined_pipeline.c -o combined_pipeline
himasree@LAPTOP-41COT5AA:~/OSLab$ gcc combined_pipeline.c -o combined_pipeline
himasree@LAPTOP-41COT5AA:~/OSLab$ ./combined_pipeline
==========================================
   COMBINED REDIRECTION & PIPELINES
==========================================

Enter combined redirection file: combined.txt
Enter pipeline output file: pipeline.txt

==========================================
 All tests completed successfully.
==========================================
himasree@LAPTOP-41COT5AA:~/OSLab$
```

```
himasree@LAPTOP-41COT5AA:~/OSLab$ cat combined.txt
This message comes from STDOUT.
This message comes from STDERR.
Both streams are redirected to the same file.
himasree@LAPTOP-41COT5AA:~/OSLab$ cat pipeline.txt
Apipes
Apipes.c
Apipes2.c
Appipes2
FIFO_CREATE
FIFO_CREATE.c
FIFO_READ
FIFO_READ.c
FIFO_WRITE
FIFO_WRITE.c
LinuxLab
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

/*
    Demonstrates:

    1. Combined stdout + stderr redirection
       command > file 2>&1

    2. Pipeline + redirection
       command1 | command2 > file
*/


/* ========================================================
   PART 1: COMBINED STDOUT + STDERR REDIRECTION
   ======================================================== */

void combined_redirection(const char *filename)
{
    int fd;

    /*
        Open file for writing.
        Existing contents are replaced.
    */
```

```
        Open file for writing.
        Existing contents are replaced.
    */
    fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0)
    {
        perror("open failed");
        return;
    }

    /*
        Redirect stdout to the file.
        stdout = file descriptor 1
    */
    if (dup2(fd, STDOUT_FILENO) < 0)
    {
        perror("dup2 stdout failed");
        close(fd);
        return;
    }

    /*
        Redirect stderr to the SAME file.
        stderr = file descriptor 2
    */
    if (dup2(STDOUT_FILENO, STDERR_FILENO) < 0)
    {
        perror("dup2 stderr failed");
        close(fd);
```

```
/* ============================================================
   PART 2: PIPE + REDIRECTION
   ============================================================ */

void pipeline_redirection(const char *filename)
{
    int pipefd[2];

    if (pipe(pipefd) < 0)
    {
        perror("pipe failed");
        return;
    }

    pid_t pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork failed");
        return;
    }

    /*
        CHILD 1
        ls output -> pipe
    */
    if (pid1 == 0)
    {
```

```
        close(pipefd[0]);

        /*
            Redirect stdout to pipe.
        */
        if (dup2(pipefd[1], STDOUT_FILENO) < 0)
        {
            perror("dup2 failed");
            exit(1);
        }

        close(pipefd[1]);

        execlp("ls", "ls", "-1", NULL);

        perror("execlp ls failed");
        exit(1);
    }

    /*
        CHILD 2
        pipe -> sort -> output file
    */

    pid_t pid2 = fork();

    if (pid2 < 0)
    {
        perror("fork failed");
        return;
```

```
        /*
            Redirect stdin from pipe.
        */
        if (dup2(pipefd[0], STDIN_FILENO) < 0)
        {
            perror("dup2 stdin failed");
            exit(1);
        }

        close(pipefd[0]);

        /*
            Open output file.
        */
        int fd = open(filename,
                      O_WRONLY | O_CREAT | O_TRUNC,
                      0644);

        if (fd < 0)
        {
            perror("open output failed");
            exit(1);
        }

        /*
            Redirect stdout to output file.
        */
        if (dup2(fd, STDOUT_FILENO) < 0)
        {
            perror("dup2 stdout failed");
```

```c
/* ============================================================
   MAIN
   ============================================================ */

int main()
{
    char combined_file[100];
    char pipeline_file[100];

    printf("=========================================\n");
    printf("   COMBINED REDIRECTION & PIPELINES\n");
    printf("=========================================\n");

    printf("\nEnter combined redirection file: ");
    scanf("%99s", combined_file);

    printf("Enter pipeline output file: ");
    scanf("%99s", pipeline_file);


    /*
        ---------------------------------------
        PART 1
        stdout + stderr -> same file
        ---------------------------------------
    */

    pid_t pid = fork();
```

```c
    /*
        ---------------------------------------
        PART 1
        stdout + stderr -> same file
        ---------------------------------------
    */

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        combined_redirection(combined_file);
        exit(0);
    }

    waitpid(pid, NULL, 0);


    /*
        ---------------------------------------
        PART 2
        ls | sort > file
        ---------------------------------------
```

```c
    */

    pipeline_redirection(pipeline_file);


    printf("\n=========================================\n");
    printf(" All tests completed successfully.\n");
    printf("=========================================\n");

    return 0;
```

```
imasree@LAPTOP-U1COI5AA:~/OSLab$
```